1	2	3	4	$\sum$

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 12

Release: 11.01.2019; Due: 25.01.2019

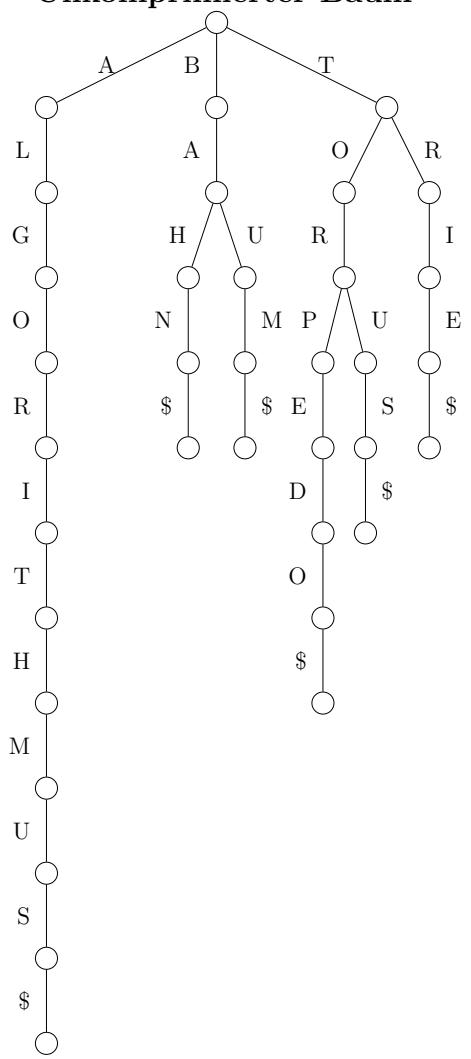
Viktoriya Kraleva, Samuel Enderwitz

---

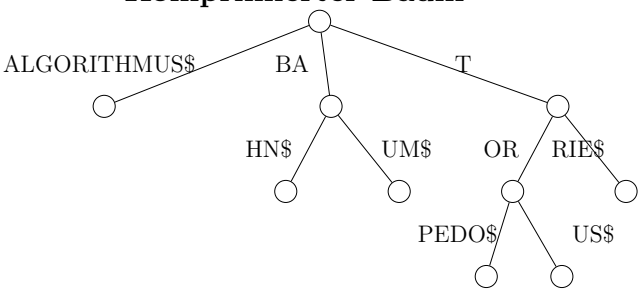
1 Tries (__/10)

(a) *Draw an uncompressed and a compressed trie for the following set of strings: {ALGORITHMUS, TRIE, BAUM, TORUS, BAHN, TORPEDO}*

Unkomprimierter Baum



Komprimierter Baum



(b) *Describe an algorithm that outputs all the strings stored in a given uncompressed trie and that visits each edge of the trie at most twice.*

Wir verwenden das Prinzip der Tiefensuche (DFS). Dabei verwenden wir eine Interative Implementierung mit einem Stack, der als 'Liste' dient, was noch getan werden muss. Dabei baut sich ein String auf, bis das Zeichen '$' erreicht wurd und gibt den String dann aus bzw. speichert ihn in einer Liste.

Eine alternative Lösung ist als Code in Aufgabe 2 zu finden.

2 Implemention of Tries (__/10)

Describe briefly, how a concrete implementation of the operations `put`(s, v), `get`(s), `remove`(s), *and* `succ`(s) *in an uncompressed and a compressed trie could be implemented. Here, s is a nonempty string, and v is a value from a set V. What are the running times of your implementations?*

Wir haben die Algorithmen erst mal so beschrieben, wie sie allgemein funktionieren, auch unabhängig von der Implementierung. Anbei haben wir noch Java-Code für einen nicht komptimierten Baum geschrieben und gehen auf den Code bezogen auf die Laufzeit ein.

Unkomprimiert

put(s,v): Gehe den String s von vorne nach hinten durch und schau, ob in dem Baum passende Kanten vorhanden sind und wenn ja, gehe diesen Weg. Falls letztes Zeichen von s auch gefunden worden ist, speichere im aktuellen Knoten den Wert v (ggf wird Wert überschrieben). Wenn ein Zeichen nicht gefunden wird, dann hänge neuen Knoten an mit dem diesem Zeichen. Nun für die reslichen Zeichen noch jeweils eine Kante anhängen und im letzten Knoten wird der Wert v gespeichert.

get(s): Gehe den String s von vorne nach hinten durch und schau, ob in dem Baum passende Kanten vorhanden sind und wenn ja, gehe diesen Weg. Falls ein Weg nicht vorhanden sein sollte, werfe eine 'NotFoundException'. Wenn man String s komplett durchgegangen ist und kein Wert im Knoten drin steht, werfe auch 'NotFoundException'. Wenn Wert im Knoten steht, gebe diesen zurück.

remove(s): Gehe den String s von vorne nach hinten durch und schau, ob in dem Baum passende Kanten vorhanden sind und wenn ja, gehe diesen Weg. Falls ein Weg nicht vorhanden sein sollte, werfe eine 'NotFoundException'. Wenn man String s komplett durchgegangen ist und kein Wert im Knoten drin steht, werfe auch 'NotFoundException'. Wenn der Knoten weitere Kinder hat, dann lösche Wert v aus dem Knoten und beende die Methode. Falls der Knoten keine weiteren Kinder mehr hat, lösche so lange Knoten dem Suchweg entlang, bis ein Wert im Knoten steht, oder der Knoten Kinder hat.

succ(s): Gehe den String s von vorne nach hinten durch und schau, ob in dem Baum passende Kanten vorhanden sind und wenn ja, gehe diesen Weg. Das machen wir so lange, bis der String s zu ende ist, oder der kein pasender Kanten gefunden wurde. Wenn der String fertig ist, gehe so lange nach 'unten links', bis ein Wert in einem Knoten steht und gebe diesen zurück. Wenn kein passender Kanten mehr gefunden wurde, dann gehe so lange nach 'unten rechts', bis ein Wert in einem Knoten steht. Falls man in einem der beiden

Fälle schon an einem Blatt angekommen ist, dann gehe den gegangen Weg so lange nach oben, bis ein 'rechteres' Kind existiert und geht dort ganz nach 'links unten'.

```java
import javax.swing.tree.TreeNode;

import java.util.ArrayList;

import static jdk.nashorn.internal.objects.Global.print;

public class UncompressedNodeT <V> {
        static final int ALPHABET_SIZE = 26;
        char key;
        V value;
        boolean isendofword;
        UncompressedNodeT<V> parent;
        UncompressedNodeT<V> children[];
        public UncompressedNodeT(){
            this.value=null;
            this.key=' ';
            this.parent=null;
            this.children=(UncompressedNodeT<V>[] )new UncompressedNodeT[ALPHABET_SIZE+1];
            this.isendofword=false;
        }

    private boolean isLeaf(UncompressedNodeT<V> node){
            return isendofword;
    }

    public String display(UncompressedNodeT<V> root,int level,ArrayList myword) {
        String mystring = new String();
        String wholelist = new String();
        int startindex = 0;
        boolean anotherkind=false;
        for (int i = 0; i < ALPHABET_SIZE + 1; i++) {
            if(root.key=='$'){
                if(!myword.isEmpty()) {
                    mystring = String.join(",", myword.toString());
                    wholelist += mystring;
                    startindex = wholelist.length();
                    mystring = new String();
                    myword=new ArrayList();
                    level=mystring.length();
                }
                myword=new ArrayList();
            }
            if (root.children[i] != null) {
                if(root.parent!=null && root.key!='\u0000' && anotherkind){
                    startindex=findstartindex(myword);
                    myword.addAll(myword.subList(startindex,level+startindex));
                    myword.add(root.children[i].key);
                    display(root.children[i], level + 1, myword);
                } else {
                    anotherkind = true;
                    myword.add(root.children[i].key);
```

```
                    //System.out.println(myword.toString());
                    display(root.children[i], level + 1, myword);
                }
                wholelist += mystring;
            }
        }
        return formatString(myword.toString());
    }

    private int findstartindex(ArrayList<Character> mylist){
            boolean second = false;
            for(int i = mylist.size()-1;i>-1;i--){
                System.out.println("(" + i + mylist.get(i).toString() + ")");
                if(mylist.get(i)=='$'&& second||(mylist.get(i)=='['&&second))
    {
                    return i+1;
                } else if (mylist.get(i)=='$'){
                    second=true;
                }
            }
            return 0;
    }

    private String formatString(String mystring){
            String formattedWord = new String();
            for(int i =0;i<mystring.length();i++){
                if(mystring.charAt(i)!=',' && mystring.charAt(i)!='$'&&
    mystring.charAt(i)!=' '){
                    formattedWord += mystring.charAt(i);
                } else if (mystring.charAt(i)=='$'&&i<mystring.length()-2){
                    formattedWord+=',';
                }
            }
            return formattedWord;
    }

    public void insert(UncompressedNodeT<V> root,String key,V value){
            UncompressedNodeT<V> helper = root;
            for(int level = 0;level<key.length();level++){
                if(helper.children[key.charAt(level)-'a']==null){
                    helper.children[key.charAt(level)-'a'] = new
    UncompressedNodeT<V>();
                    helper.children[key.charAt(level)-'a'].key=key.charAt(
    level);
                    helper.children[key.charAt(level)-'a'].parent=helper;
                }
                helper =  helper.children[key.charAt(level)-'a'];
            }
            if(helper.children[ALPHABET_SIZE]==null){
                helper.children[ALPHABET_SIZE]=new UncompressedNodeT<>();
                helper.children[ALPHABET_SIZE].value=value;
                helper.children[ALPHABET_SIZE].parent = helper;
                helper.children[ALPHABET_SIZE].key='$';
                helper.children[ALPHABET_SIZE].isendofword=true;
            }
        }

```

```java
        public void remove(UncompressedNodeT<V> root, String key, int charAt)
    {
            if(root == null){
                print("Tree is empty");
            } else if(charAt!=key.length()){
                if(!isEmpty(root)){
                    remove(root.children[key.charAt(charAt)-'a'],(String) key
    ,charAt+1);
                    delete(root);
                }else {
                    print("Such word does not exist");
                }
            } else if (charAt==key.length()){
                //System.out.println(root.children[ALPHABET_SIZE]);
                if(root.children[ALPHABET_SIZE]==null){
                    System.out.println("Such word does not exist");
                } else {
                    delete(root.children[ALPHABET_SIZE]);
                    delete(root);
                    root = null;
                    }
                }
            }

        public boolean isEmpty(UncompressedNodeT<V> root){
            for(int i =0;i<ALPHABET_SIZE;i++){
                if(root.children[i]!=null){
                    return false;
                }
            }
            return true;
        }

    public UncompressedNodeT<V> getNode(UncompressedNodeT<V> root,String key)
    {
        for(int i = 0;i<key.length();i++){
            int index = key.charAt(i)-'a';
            if(root.children[index]!=null){
                root = root.children[index];
            } else {
                print("Such element does not exist");
                return null;
            }
        }
        if(root.children[ALPHABET_SIZE]!=null){
            return root.children[ALPHABET_SIZE];
        } else {
            print("Such element does not exist");
            return null;
        }
    }

        public V get(UncompressedNodeT<V> root,String key){
            for(int i = 0;i<key.length();i++){
                int index = key.charAt(i)-'a';
                if(root.children[index]!=null){
                    root = root.children[index];
```

```
                } else {
                    print("Such element does not exist");
                    return null;
                }
            }
            if(root.children[ALPHABET_SIZE]!=null){
                return root.children[ALPHABET_SIZE].value;
            } else {
                print("Such element does not exist");
                return null;
            }
        }

        /*SUCCESSOR funktioniert,aber mann kann entweder dieser Ansatz verwenden,oder
die Graph als eine Liste ausgeben und zwar mit allen Elementen,damit ware es super easy,wenn man die Elemente in dem
Array sortiert und damit den Nachfolger findet*/

    public UncompressedNodeT<V> inOrderSuccessor(UncompressedNodeT<V> root, String key) {
            UncompressedNodeT<V> mynode = getNode(root,key);
            if(!isEmpty(mynode)){
                for(int i = 0;i<ALPHABET_SIZE;i++){
                    if(!isEmpty(mynode)){
                        return minValue(mynode.children[i]);
                    }
                }
            }
            mynode = getNode(root,key).parent;
            for(int i = mynode.key-'a'+1;i<ALPHABET_SIZE;i++){
                if(!isEmpty(mynode)){
                    return minValue(mynode.children[i]);
                }
            }

            UncompressedNodeT<V> p = mynode.parent;
            while (p!=null && mynode==p.children[mynode.key-'a']){
                for(int i = mynode.key+1 -'a';i<ALPHABET_SIZE;i++){
                    if(p.children[i]!=null){
                        return minValue(p.children[i]);
                    }
                    if(p.parent==null){
                        break;
                    }
                    mynode=p;
                    p=p.parent;
                }
                for(int i = mynode.key+1 -'a';i<ALPHABET_SIZE;i++) {
                    if (p.children[i] != null) {
                        return minValue(p.children[i]);
                    }
                }
            }
            return null;
        }

```

```java
        private UncompressedNodeT<V> minValue(UncompressedNodeT<V> node){
            UncompressedNodeT<V> current = node;
            while (!isEmpty(current)){
                for(int i=0;i<ALPHABET_SIZE;i++){
                    if (current.children[i]!=null){
                        current=current.children[i];
                        break;
                    }
                }
            }
            if(current!=node) {
                return current.children[ALPHABET_SIZE];
            } else {
                return current;
            }
        }


    private UncompressedNodeT<V> letzteGemKnoten(UncompressedNodeT<V> root,
    String key1,String key2){
            for(int i = 0;i<Math.min(key1.length(),key2.length());i++){
                if(key1.charAt(i)==key2.charAt(i)){
                    root = root.children[key1.charAt(i)-'a'];
                } else{
                    break;
                }
            }
            return root;
        }

        private V getModified(UncompressedNodeT<V> root,String key){
            for(int i = 0;i<key.length();i++){
                int index = key.charAt(i)-'a';
                if(root.children[index]!=null){
                    root = root.children[index];
                } else {
                    print("Such element does not exist");
                    return null;
                }
            }
            if(root.children[ALPHABET_SIZE]!=null){
                return root.children[ALPHABET_SIZE].value;
            } else {
                print("Such element does not exist");
                return null;
            }
        }
        private void delete(UncompressedNodeT<V> node){
            if(node.key=='$') {
                node.parent.children[ALPHABET_SIZE] = null;
            }else if ((node.parent!=null && node.parent.key!='\u0000')){
                node.parent.children[node.key-'a'] = null;
            }
            node=null;
            }

    public static void main(String[] args){
```

```
        UncompressedNodeT mytrie = new UncompressedNodeT();
        mytrie.insert(mytrie,"kotka",3);
        mytrie.insert(mytrie,"kotak",5);
        mytrie.insert(mytrie,"mase",4);
        mytrie.insert(mytrie,"mordor",4);
        ArrayList mylist = new ArrayList();
        System.out.println(mytrie.display(mytrie,0,mylist));
       // mytrie.remove(mytrie,"kotka",0);
        mylist = new ArrayList();
        System.out.println(mytrie.display(mytrie,0,mylist));
        System.out.println(mytrie.inOrderSuccessor(mytrie,"kotka").value);
    }

}
```

Listing 1: UncompressedNode

Komprimiert

Ein komptimierter Trie funktioniert im wesentlichen genau gleich wie ein unkomprimierter. Wir beschreiben deshalb nur die unterschiede erneut.

put(s,v): Der Vorgang die Stelle zu finden, an dem er einsortiert wird, funktioniert ein wenig anders. Da wir nicht immer nur ein Zeichen gespeichert haben, müssen wir prüfen, ob der gesamte String, der im Knoten gespeichert ist, ein Präfix des Teilstrings ist, den wir aktuell haben. Falls das nicht der Fall sein sollte, müssen wir diesen Kanten splitten und ein Knoten dazwischen schieben. Dabei hat die obere Kante den Teilstring mit den gleichen Zeichen am Anfang. Der Knoten bekommt zwei Kinder, mit den Kanten des restlichen Teilsstrings.

get(s): Der Vorgang die Stelle zu finden, an dem er einsortiert wird, funktioniert ein wenig anders. Da wir nicht immer nur ein Zeichen gespeichert haben, müssen wir prüfen, ob der gesamte String, der im Knoten gespeichert ist, ein Präfix des Teilstrings ist, den wir aktuell haben.

remove(s): Der Vorgang die Stelle zu finden, an dem er einsortiert wird, funktioniert ein wenig anders. Da wir nicht immer nur ein Zeichen gespeichert haben, müssen wir prüfen, ob der gesamte String, der im Knoten gespeichert ist, ein Präfix des Teilstrings ist, den wir aktuell haben.

succ(s): Eins nach oben und dann das "nächste" Kind.

Laufzeiten unkomprimierter Trie:

display: unserer Meinung nach ist die Zeitliche Komplexität $n \cdot |E| \cdot maxlength \in O(n \cdot |E| \cdot maxlength)$ Weil alle Wörter sich im schlimmsten Fall bei einer Buchstabe unterscheiden und n ist die Anzahl der Wörter. Dafür haben wir jeweils 1 Durchlauf für jede Buchstabe, die gefunden wird, und im Schlimmsten fall können diese alle Wörter in n Plätzen unterscheiden.

get(): offensichtlich linearer Laufzeit $O(|w|)$, wo $|w|$ die Wortlänge ist

put(): die Laufzeit ist nochmal offensichtlich linear ein schleifendurchlauf für jede Buchstabe einen Durchlauf (n-Länge des Worts) $O(n)$

remove(): Wir rufen die Funktion jeweils rekursiv für jedes Zeichen des Worts, alle Operationen aber haben konstanten Laufzeit. Deswegen haben wir eine Lineare Laufzeit, die von der Länge des Worts abhängig ist und zwar (n-Länge des Worts) $O(n)$

succ(): Maximal $|w_1|$ Mal nach oben (w_1 ist unserer Eingabe Wort) und maximal $|w_2|$ Mal nach unten. (w_2 ist der Nachfolgerwort). Jeweils $|E|$ Schleifendurchlaufe, deswegen $O((w_1 + w_2) \cdot |E|)$

Laufzeiten komptimierter Trie:

display: Am schlimmsten, sind die Wörter so ähnlich, dass der Baum wie ein uncompressed Trie aussieht, damit haben wir am schlimmsten die Komplexität $n \cdot |E| \cdot maxlength \in O(n \cdot |E| \cdot maxlength)$
get(): linearer Laufzeit $O(|w|)$, für jede Buchstabe des Worts bestimmte Anzahl an Elementaroperationen

put(): nochmal am schlimmsten, wenn es genauso wie einen uncompressed Baum aussieht, weil die Wörter sehr nah zueinander sind. Damit auch die selbe worst-case Laufzeit: $O(n + T_{(stringVergleich)})$. Mann mus hier auch nun merken, dass die Stringvergleichsoperationen auch einige Einheiten kosten und davon hängt dies auch ab. Wir wissen nicht genau, wie diese in Java oder eine andere Programmiersprache genau implementiert sind, deswegen haben wir diese weggelassen.

remove(): Nochmal wie oben, aber muss man auch die zeitliche Komplexität der Stringvergleichsoperationen in Kauf nehmen. $O(n + T_{(stringVergleich)})$

succ(): Hier haben wir ein Unterschied und zwar, dass wir nur dann splitten, wenn es 2 Wörter gibt. Damit können wir immer bei dem Elternknoten sicher sein, dass er mehr als ein Kind hat und damit auch den Nachfolger in linearer Zeit finden. Wir bennenen $|w_{suffixNachfolger}|$ der Teilstring des Nachfolgers, die am unterschiedlich von unserem ist. Damit können wir maximal $|E| \cdot |w_{suffixNachfolger}| + 1$ Elementaroperationen haben, wo k die Anzahl der Knoten bei dieser Teilstring ist (mann kann dies nicht wissen). Damit haben wir $|E| \cdot w_{suffixNachfolger} + 1 \in O(|E| \cdot |w_{suffixNachfolger}|)$ und dies ist schon einen Unterschied.

3 Depth-First Search (__/10)

(a) *Use depth-first search in order to find a walk in a connected, undirected, simple graph that visits each edge exactly once in each direction.*

Bei der Tiefensuche wird jeder Knoten nur einmal besucht. Dabei kann man sagen, dass der Algorithmus erstmal einen so tiefen Weg wie möglich geht und dann den nächsten Strang geht. Wir passen den DFS Algorithmus so an, dass wir vor und nach dem rekursiven Aufruf wir die Kante in einer Liste speichern. Dabei ist des vor dem Aufruf, wenn wir den Weg in einer Richtung gehen und der nach dem Aufruf, wenn wir wieder zurück gehen.
Wir müssen den Algorithmus aber noch weiter bearbeiten, da wenn ein Knoten schon besucht wurde, der Weg nicht gegangen wird. Diesen Weg gehen wir hin und direkt wieder zurück.

```
A <- new List
dfs_walk(v):
    for e in v.outgoingEdges() do
        w <- e.opposite(v)
        if !w.found then
            w.found = true
            A.append(e)
            dfs_walk(w)
            A.append(e)
        else // that it work with graphs with circles
            A.append(e)
            A.append(e)
```

Listing 2: Pseudocode für Wegefindung

(b) *Let $G = (V, E)$ be a directed acyclic graph (DAG). A topological ordering of G is an ordering of the vertices of G such that no edge in G goes from a later to an earlier vertex in the ordering. Consider the following algorithm.*

Bilder sind im Anhang.

Setze alle 'found' auf false
Gehe alle Knoten in aufsteigender Reihenfolge durch
  1 schon 'found'? Nein; rufe dfs auf
  1 nun 'found'
  Gehe alle ausgehenden Kanten in aufsteigender Reihenfolge durch
    3 noch nicht gefunden, rufe 'dfs' rekursiv auf.
      3 nun 'found'
      Gehe alle ausgehenden Kanten in aufsteigender Reihenfolge durch
        2 noch nicht gefunden, rufe 'dfs' rekursiv auf.
          2 nun 'found'
          Gehe alle ausgehenden Kanten in aufsteigender Reihenfolge durch
            5 noch nicht gefunden, rufe 'dfs' rekursiv auf.
              5 nun 'found'
              Keine ausgehenden Kanten
            pushe 5
          pushe 2
        5 bereits gefunden
      pushe 3
  pushe 1
  2 schon 'found'? Ja
  3 schon 'found'? Ja
  4 schon 'found'? Nein
  4 nun 'found
  Gehe alle ausgehenden Kanten in aufsteigender Reihenfolge durch
    5 bereits gefunden
  pushe 4
  5 schon 'found'? Ja
  6 schon 'found'? Nein
  6 nun 'found

Gehe alle ausgehenden Kanten in aufsteigender Reihenfolge durch
4 bereits gefunden
5 bereits gefunden
pushe 6

Daraus folgt die Topologische Reihenfolge von: $6, 4, 1, 3, 2, 5$